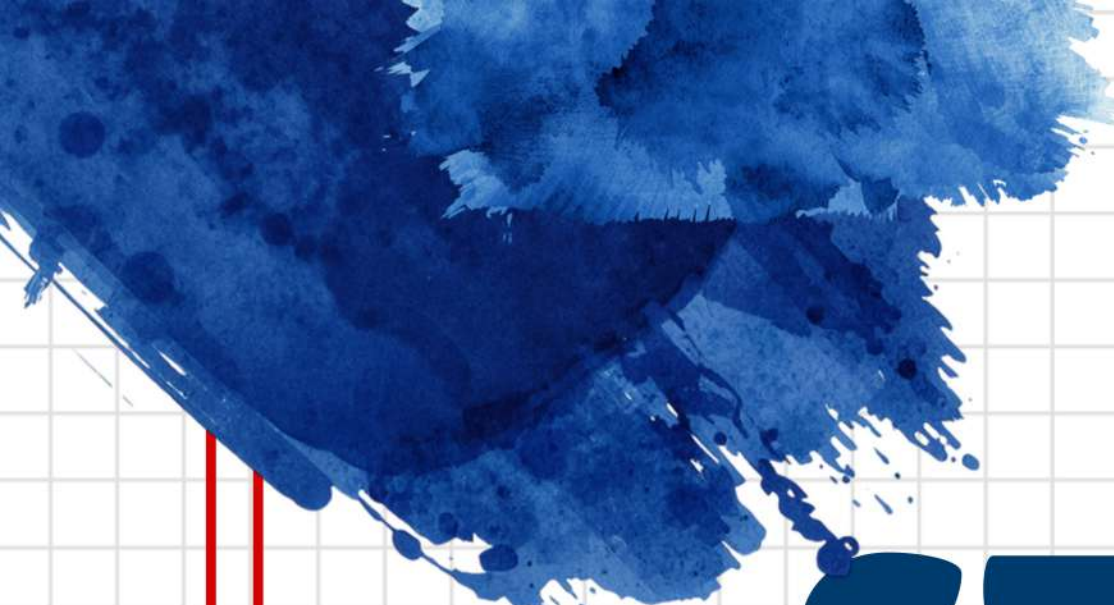
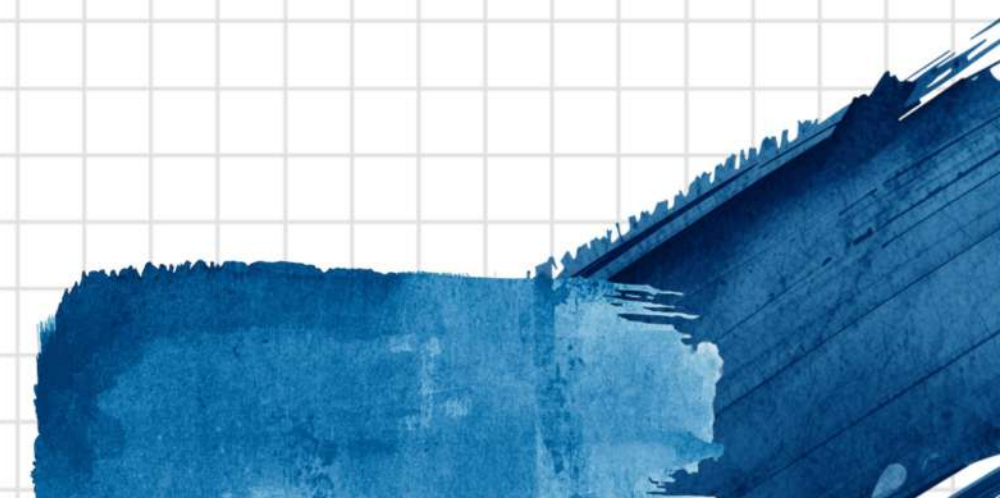




STOCK BOOST

DISEÑO Y DESARROLLO DE BASE DE DATOS RELACIONAL



CONTEXTO DEL PROYECTO

Stock Boost surge como respuesta a la necesidad de optimizar la gestión de inventario y ventas en tiendas de barrio. El sistema busca resolver problemas críticos como el control de stock, la identificación de productos próximos a vencer y la toma de decisiones informadas sobre reposición de mercancía.

El objetivo principal es proporcionar una base de datos relacional robusta que permita a los pequeños comerciantes tomar decisiones estratégicas basadas en datos confiables, reduciendo pérdidas y maximizando la rentabilidad de su negocio.

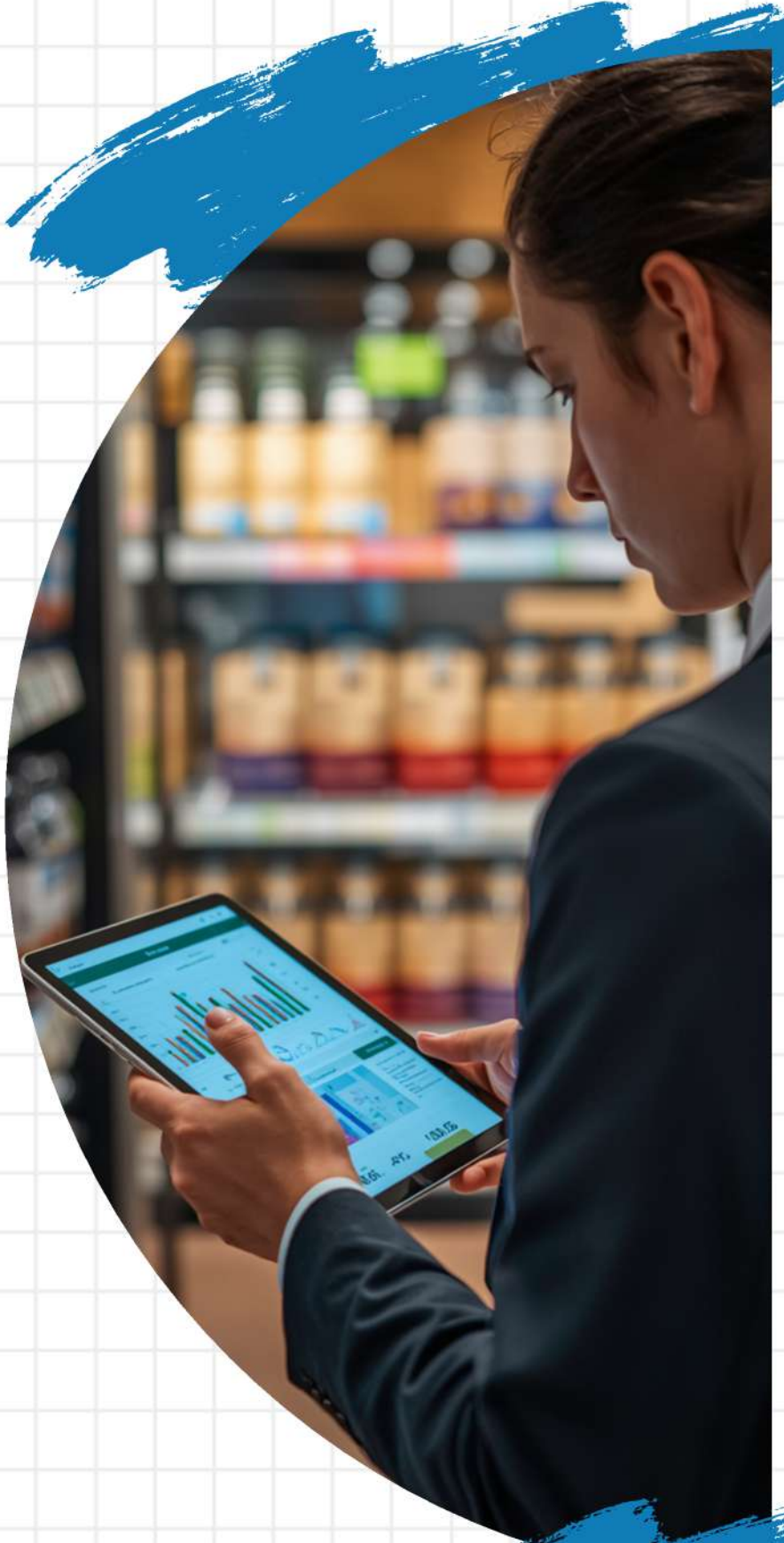


HISTORIAS DE USUARIO PRIORITARIAS

- ✓ RQF-01: REGISTRO DE PRODUCTOS
- ✓ RQF-02: VENTAS AUTOMÁTICAS
- ✓ RQF-03: RECEPCIÓN PEDIDOS
- ✓ RQF-04: ALERTAS STOCK BAJO
- ✓ RQF-05: REPORTES VENTAS

FUNCIONALIDADES PENDIENTES

RQF-06: Top 10 productos vendidos
RQF-07: Alertas de vencimiento
RQF-08: Consulta margen ganancia



HISTORIAS DE USUARIO

6

RQF-06: Top 10 Productos Más Vendidos

Como dueño, quiero ver los 10 productos más vendidos para identificar tendencias y optimizar el inventario según la demanda real.

7

RQF-07: Alertas por Proximidad de Vencimiento

Como empleado, necesito recibir alertas de productos próximos a vencer (± 15 días) para para hacer una evaluacion objetiva de que se debe de hacer.

8

RQF-08: Consulta de Margen de Ganancia

Como administrador, quiero consultar el margen de ganancia por producto para evaluar la rentabilidad y ajustar precios estratégicamente.

ARQUITECTURA DEL MODELO

BASE DE DATOS

Capa de
Referencia:
categorías,
proveedores,
usuarios

Capa
Transaccional:
productos,
ventas, pedidos

Capa de
Auditoría:
historial_precios,
logs

Integridad y
trazabilidad
garantizadas

TABLAS MAESTRAS – CAPA DE REFERENCIA

CATEGORÍAS

10 grupos de productos predefinidos para clasificación. Permite organizar el inventario por tipo: alimentos, bebidas, aseo, etc. Campo UNIQUE en nombre para evitar duplicados.

PROVEEDORES

Datos de contacto completos: nombre, teléfono, email, dirección. Relación 1:N con productos y pedidos. Permite trazabilidad de origen y gestión de órdenes de compra.

USUARIOS

Control de acceso con roles ENUM: admin, empleado, dueño. Cada rol define permisos específicos en el sistema. Garantiza integridad y seguridad en operaciones críticas.

TABLAS TRANSACCIONALES – CAPA CENTRAL

PRODUCTOS

Tabla central con atributos clave: id_producto, nombre, precios, stock_actual, stock_minimo, fecha_vencimiento. Incluye FK a categorías y proveedores para mantener integridad referencial.

VENTAS + DETALLE_VENTA

Patrón cabecera-detalle: ventas almacena id_venta, fecha, total y estado. detalle_venta resuelve relación N:M con productos, registrando cantidad, precio_unitario y subtotal calculado.

PEDIDOS + DETALLE_PEDIDO

Gestión de órdenes a proveedores: pedidos registra id_proveedor, fechas y estado. detalle_pedido especifica productos y cantidades con costo de adquisición para actualización de stock.

TABLAS DE AUDITORÍA – CONTROL Y SEGUIMIENTO

historial_precios: Registra automáticamente cada cambio en los precios de productos mediante triggers. Permite análisis histórico de márgenes y trazabilidad completa de modificaciones para auditorías financieras.

auditoria_movimientos: Log centralizado que captura operaciones críticas del sistema (ventas, devoluciones, ajustes de stock). Esencial para control de calidad, detección de anomalías y cumplimiento de seguridad.



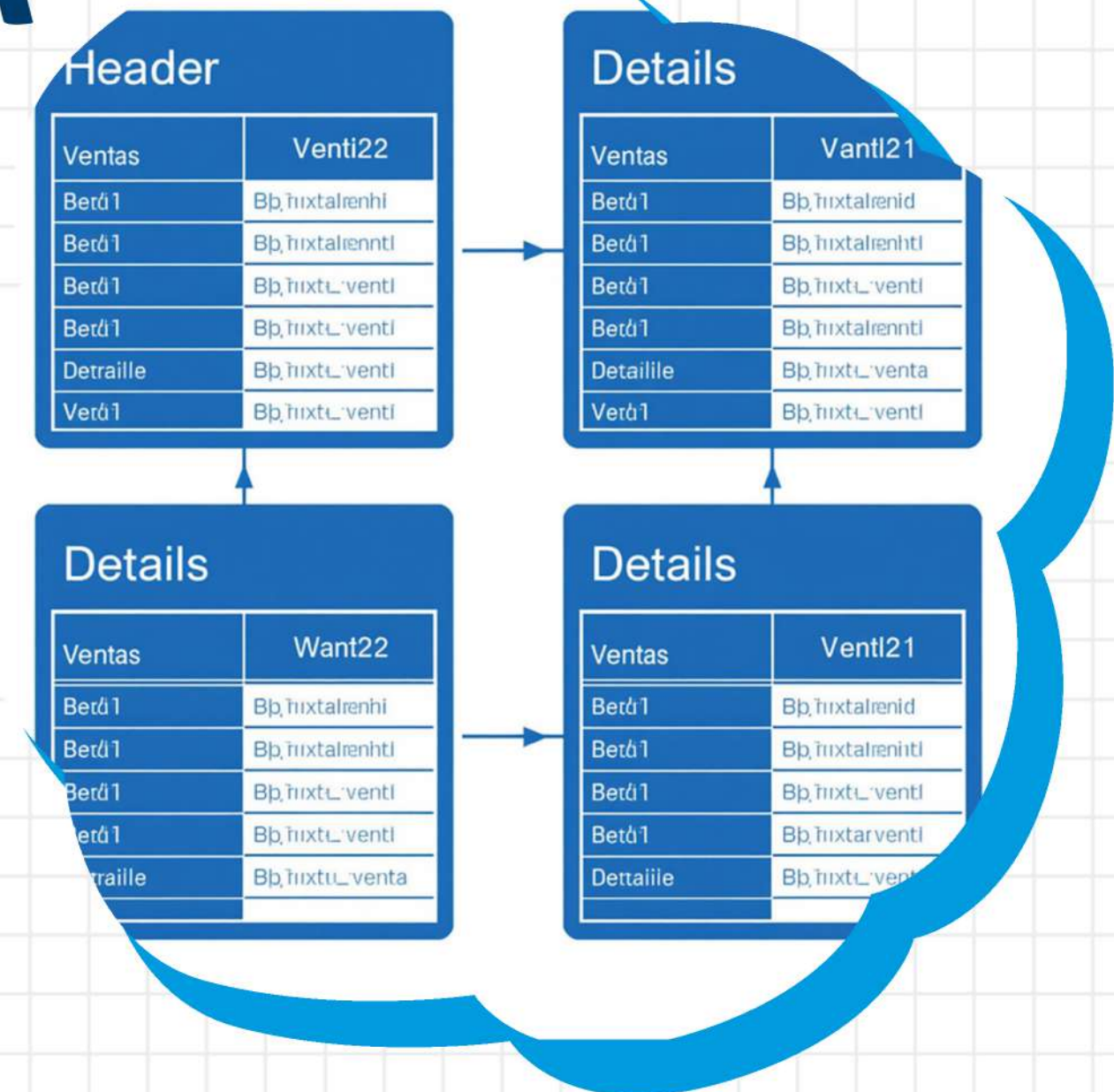
PROFUNDIZACIÓN EN VENTAS Y DETALLE_VENTA

Tabla ventas (Cabecera):

- id_venta: INT AUTO_INCREMENT (PK)
- id_usuario: FK al empleado que registra
- fecha: DATETIME de la transacción
- total: DECIMAL calculado automáticamente
- estado: ENUM ('completada', 'cancelada')

Tabla detalle_venta (Detalle):

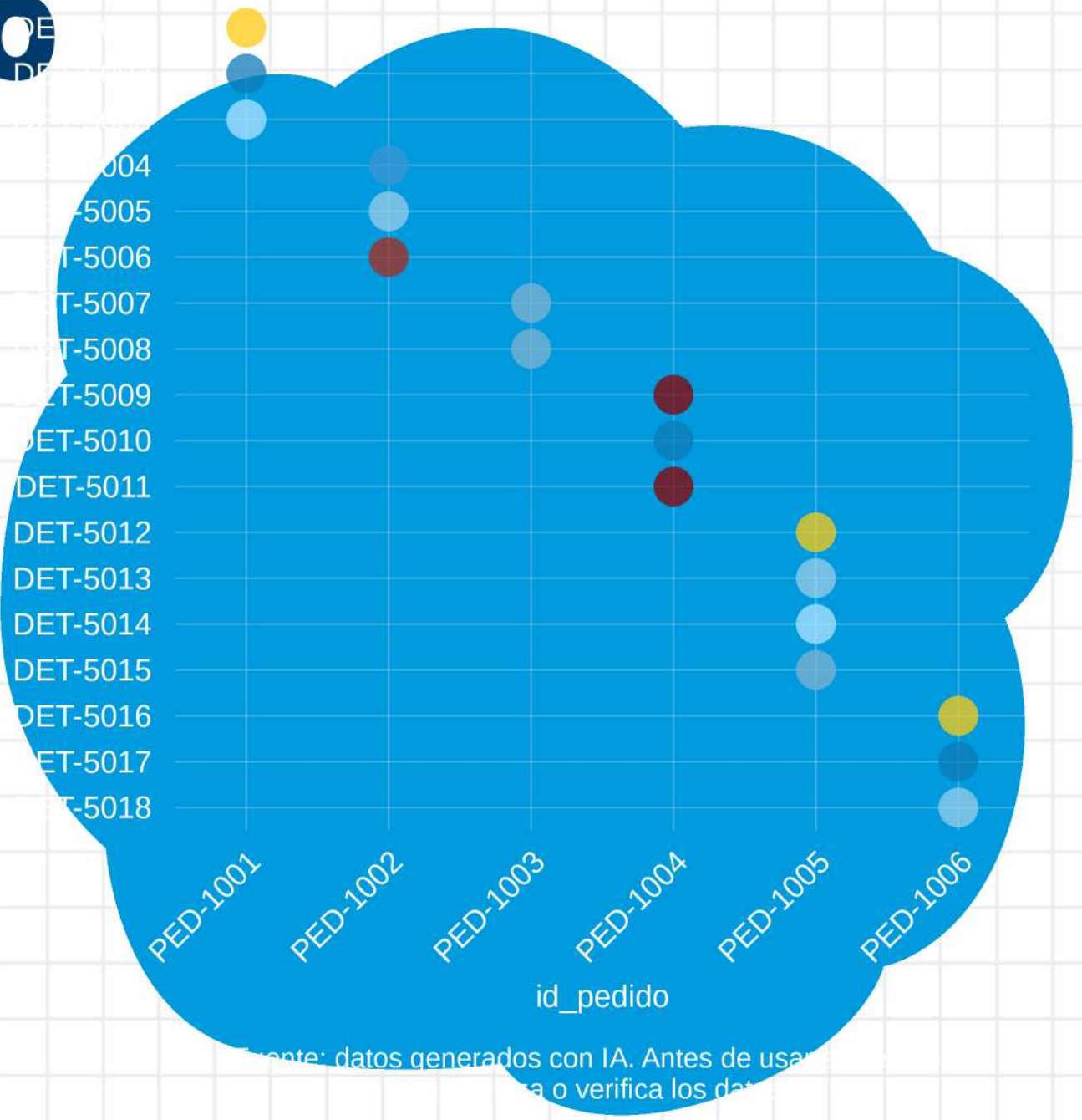
- id_producto: FK con cantidad vendida
- precio_unitario: precio al momento de venta
- subtotal: GENERATED ALWAYS AS STORED
- Resuelve relación N:M productos-ventas
- Triggers garantizan integridad y consistencia



PROFUNDIZACIÓN EN PEDIDOS Y DETALLE_PEDIDO

Tabla pedidos: Registra órdenes de compra a proveedores con id_pedido (PK), id_proveedor (FK), id_usuario (FK), fecha_pedido, fecha_entrega, total (DECIMAL) y estado (ENUM: pendiente, recibido, cancelado).

Tabla detalle_pedido: Resuelve relación N:M entre pedidos y productos. Incluye id_detalle, id_pedido (FK), id_producto (FK), cantidad_solicitada y costo_adquisicion para calcular valor total del pedido.



● RESTRICCIONES DE INTEGRIDAD Y TIPOS CLAVE

- ✓ CHECK PRECIO $\geq$ COMPRA
- ✓ CHECK CANTIDAD > 0
- ✓ UNIQUE EN CAMPOS
- ✓ ENUM ESTADOS/ROLES
- ✓ TIPOS DE DATOS SQL

MALAS PRÁCTICAS

Evitar: campos sin validación, tipos genéricos, duplicados en tablas maestras, estados como texto libre.

IMPLEMENTACIÓN SQL – ESTRUCTURA DEL SCRIPT

SECCIÓN 2: DDL

Definición de estructura con ordenación por dependencias. Primero tablas maestras (categorías, proveedores, usuarios), luego transaccionales (productos, ventas, pedidos) y finalmente auditoría.

SECCIÓN 3: DML

Población de datos realistas: 50 productos diversos con precios y stock, 60 ventas simuladas con sus detalles, pedidos a proveedores y registros de auditoría para pruebas completas.

SECCIONES 4-6: CONSULTAS

Consultas específicas por tabla, consultas multi-tabla con JOINS complejos, y subconsultas avanzadas para reportes de inventario, ventas por período y análisis de márgenes.

CONSULTAS Y PROCEDIMIENTOS

1

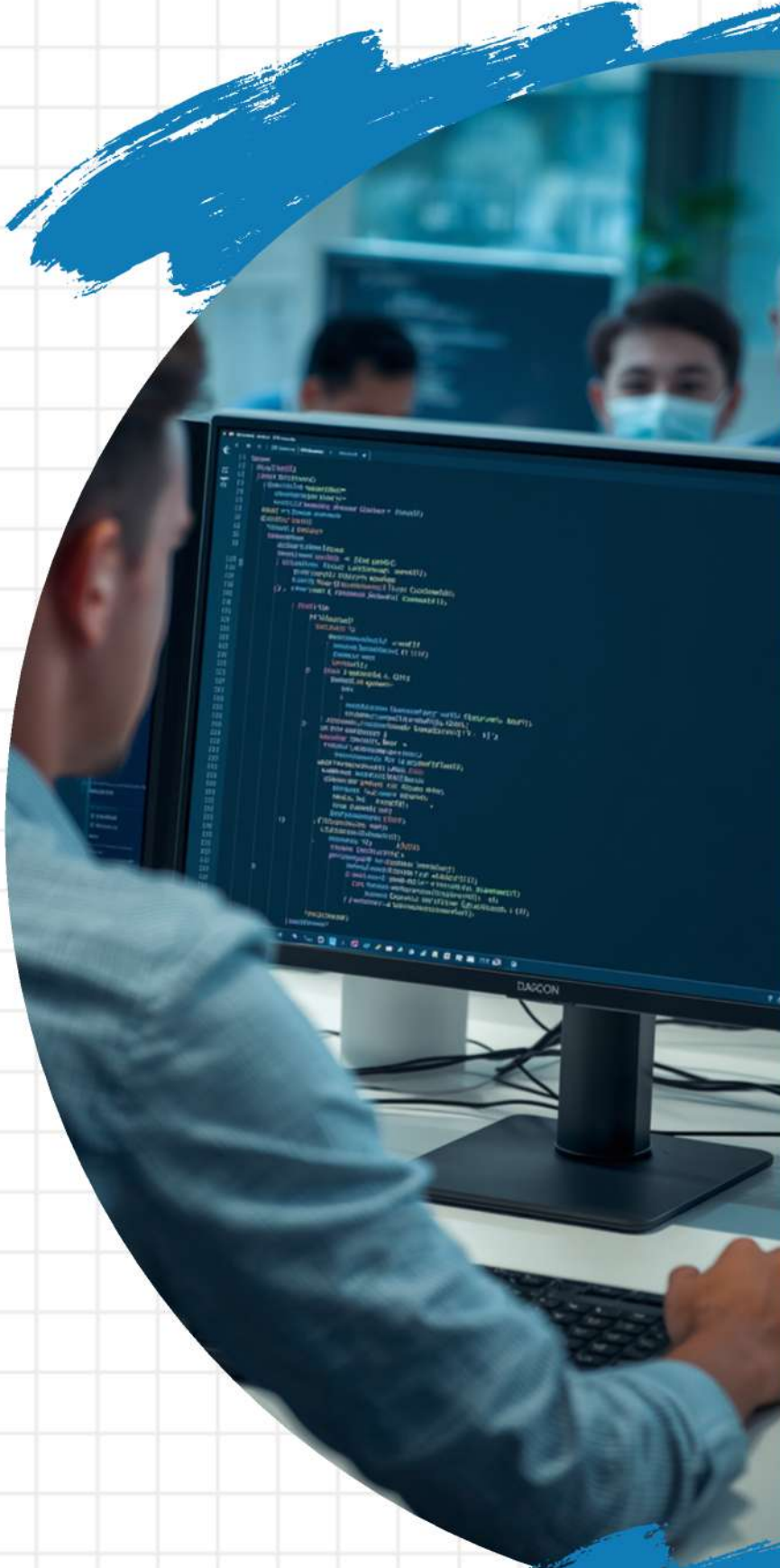
`sp_registrar_venta`: Procedimiento que ejecuta transacción completa de venta con control de errores mediante ROLLBACK. Valida stock disponible, actualiza inventario y registra detalles en patrón cabecera-detalle.

2

`sp_recibir_pedido`: Actualización iterativa del inventario al recibir mercancía. Recorre cada línea del pedido, incrementa `stock_actual` de productos y marca el pedido como recibido con fecha de recepción.

3

`sp_reporte_ventas`: Genera reportes por rango de fechas para KPIs diarios. Calcula totales, cantidad de transacciones, ticket promedio y productos más vendidos para análisis de rendimiento.



CASO DE USO REAL: TIENDA DE DON PEPE

Ejemplo práctico del flujo operativo diario:

- Inicio del día: Consulta de alertas de stock bajo y productos próximos a vencer desde las vistas del sistema
 - Durante la jornada: Registro de ventas mediante `sp_registrar_venta` con control automático de inventario
 - Cierre diario: Generación de reportes de ventas y análisis de KPIs con integración a Power BI
- Este flujo demuestra cómo Stock Boost optimiza la gestión integral del negocio.





CALIDAD DE DATOS Y AUDITORÍA

1

Validaciones en motor de base de datos: Restricciones CHECK para reglas de negocio ($\text{precio_venta} \geq \text{precio_compra}$), campos GENERATED ALWAYS AS para cálculos automáticos de subtotales y totales.

2

Triggers de auditoría: `trg_historial_precios` captura cambios en precios, `trg_auditoria_movimientos` registra operaciones críticas antes de modificaciones o eliminaciones.

3

Patrón cabecera-detalle: Estructura `ventas/detalle_venta` y `pedidos/detalle_pedido` garantiza consistencia transaccional e integridad referencial en todas las operaciones.

JUSTIFICACIÓN DEL DISEÑO

NORMALIZACIÓN 2FN

Separación de ventas y detalle_venta elimina redundancia de datos. Cada tabla tiene propósito único: cabecera con metadatos, detalle con líneas individuales. Facilita consultas y mantiene integridad referencial.

HISTORIAL DE PRECIOS

Tabla historial_precios conserva registro temporal de cambios. Permite análisis de márgenes históricos y auditoría de modificaciones. Trigger automático captura cada actualización de precio.

LOG CENTRALIZADO

Auditoria_movimientos como registro único de operaciones críticas. Consolida trazabilidad de ventas, pedidos y devoluciones. Simplifica reportes de seguridad y análisis forense de datos.

RESUMEN Y PRÓXIMOS PASOS

El sistema Stock Boost presenta un diseño robusto y funcional para la gestión integral de inventarios en tiendas de barrio. Como próximos pasos se contempla la integración con datos reales y calibración avanzada del sistema, así como la posible extensión hacia procesos ETL y análisis predictivo para optimizar decisiones de stock.